

Relative path: contracts/schemas.py
Filename: schemas.py Extension: .py

"""

OASIS v2 – shared data contracts.

Every layer imports its input/output shapes from here. Nobody is allowed to silently drift a field name mid-build – if a layer needs a new field, it gets added here first, and every downstream layer that consumes it gets updated in the same commit.

NOTE on `label` / `anomaly\_type`: these fields exist ONLY on the ground-truth side channel (see data_gen/generate_logs.py -> ground_truth.json). They are never present on the model-facing Session objects the detection/classification layers actually see. AccessEvent/Session below intentionally have no label field for that reason.

"""

from __future__ import annotations

from enum import Enum

from typing import List, Optional

from pydantic import BaseModel, Field

class EntityType(str, Enum):

 user = "user"

 service_account = "service_account"

 edge_device = "edge_device"

class AttackType(str, Enum):

 """The 7 required attack types + 1 edge case (insider_drift), per the brief's suggested taxonomy and ARCHITECTURE.md Section 1/3."""

 brute_force = "brute_force"

 impossible_travel = "impossible_travel"

 credential_stuffing = "credential_stuffing"

 lateral_movement = "lateral_movement"

 low_and_slow = "low_and_slow"

 privilege_escalation = "privilege_escalation"

 device_spoofing = "device_spoofing"

 # Edge case: NOT an attack. A legitimate behavioral shift (new work
 # pattern / new device) used to test that concept drift does not result
 # in a permanently-flagged entity. Ground truth label = "normal".

 insider_drift = "insider_drift"

class AccessEvent(BaseModel):

 """One raw access/connection event. Matches the brief's suggested schema."""

 event_id: str

 entity_id: str

 entity_type: EntityType

 timestamp: str # ISO8601

 source_ip: str

 geo_location: str

 resource_accessed: str

 auth_method: str

 action: str

 auth_success: bool

 device_fingerprint: str

 session_id: str

class Session(BaseModel):

 """The unit of analysis (ARCHITECTURE.md Section 1). One login-to-logout span per entity. This is what both the profiling and detection models actually score. NO label field on purpose – ground truth lives only in ground_truth.json."""

 session_id: str

 entity_id: str

 entity_type: EntityType

 start_time: str

 end_time: str

 session_duration_sec: float

 command_sequence: List[str] # ordered actions taken

 resources_touched: List[str]

```

auth_failures: int
source_ips: List[str]
geo_locations: List[str]
device_fingerprint: str
is_new_device: bool

class AnomalyEvent(BaseModel):
    """Output of Layer 0 (profiling, rarity_score) + Layer 1 (detection,
    anomaly_score). Combined here because the detection layer's fallback path
    (cold-start rule engine) IS the profiling layer's cold-start signal."""
    session_id: str
    entity_id: str
    anomaly_score: float          # 0-1, higher = more anomalous
    rarity_score: float          # 0-1, from baseline profiler
    is_cold_start: bool
    used_fallback_rule: bool      # True if scored by rule engine, not GRU
    contributing_features: List[str] # ordered, most-important first

class ClassifiedAnomaly(BaseModel):
    """Output of Layer 2. Only ever computed for sessions Layer 1 flagged."""
    session_id: str
    anomaly_type: AttackType
    type_confidence: float
    class_probabilities: dict

class RiskAssessment(BaseModel):
    """Output of Layer 3. Transparent arithmetic only – no model here."""
    session_id: str
    entity_id: str
    anomaly_score: float
    anomaly_type: AttackType
    type_confidence: float
    mitre_technique_id: Optional[str]
    mitre_technique_name: Optional[str]
    asset_criticality: int
    impact_score: float          # 0-1
    contributing_features: List[str]
    recommended_action: str      # acknowledge / isolate / escalate

class IncidentNarrative(BaseModel):
    """Output of Layer 4 (not built yet – placeholder contract so Layer 3's
    output shape is already the exact thing Layer 4 will consume)."""
    session_id: str
    summary: str
    why_flagged: str
    recommended_action: str
    confidence: float

```